

## Challenge #1

### 1. Define the problem

As mentioned in the module, the problem has been defined: How can we identify countries with the biggest health tech market sizes?

### 2. Get the Data

The screenshot shows the World Bank DataBank interface. On the left, under 'Variables', the 'Database' is set to 'World Development Indicators'. Under 'Country', 'Afghanistan' is selected. Under 'Series', two series are selected: 'Current health expenditure (% of GDP)' and 'Current health expenditure per capita (current US\$)'. The 'Preview' section on the right shows the data for Afghanistan in 2018. The table has columns for the country, the year, and the two selected series. The values are 14.21 for the percentage of GDP and 71.33 for the per capita expenditure.

|                                                      | 2018  |
|------------------------------------------------------|-------|
| Current health expenditure (% of GDP)                | 14.21 |
| Current health expenditure per capita (current US\$) | 71.33 |

Then, I downloaded the data according to the desired data type; here, I downloaded the data in Excel format.

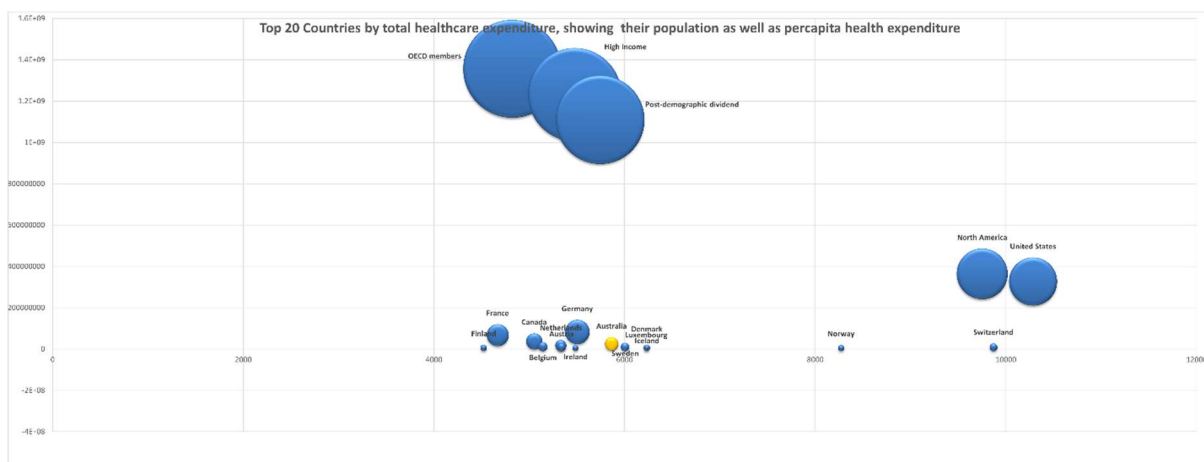
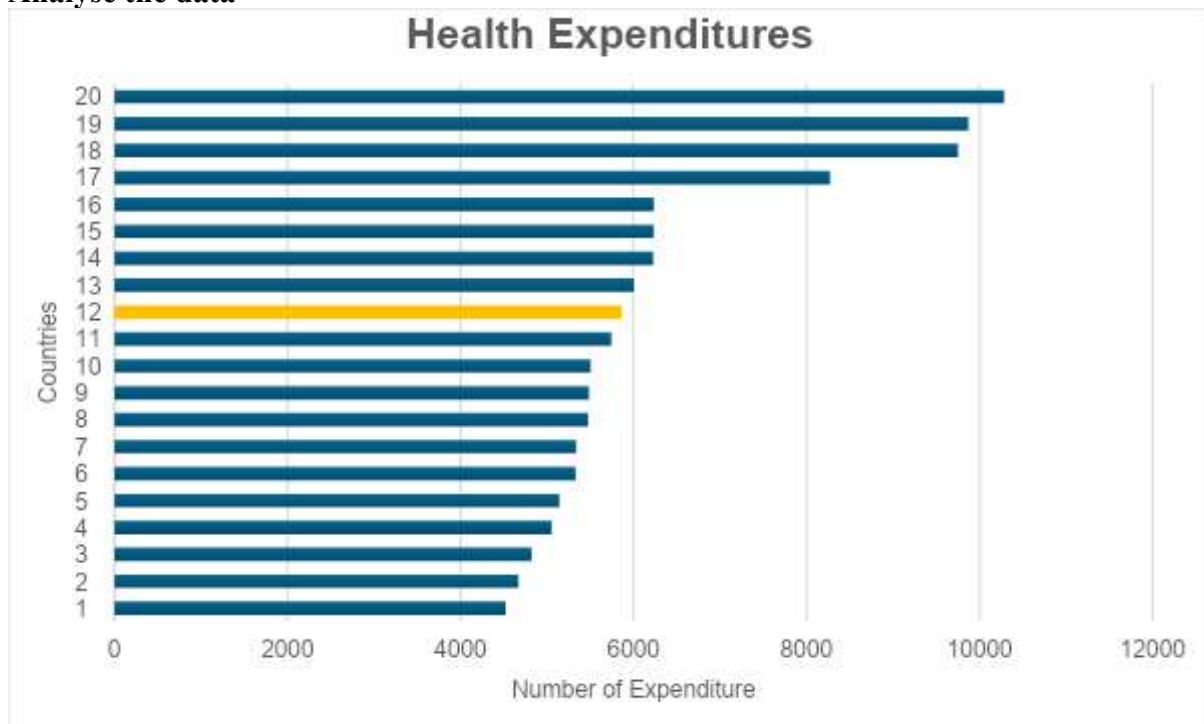
### 3. Clean and manipulate the data

There is missing data in the 2018 total expenditure column, we can drop it by applying a filter on all columns then unchecking the missing data.

The screenshot shows an Excel spreadsheet with a filter applied to the 2018 total expenditure column. The filter menu is open, showing options like 'Sort Smallest to Largest', 'Sort Largest to Smallest', 'Sort by Color', 'Sheet View', 'Clear Filter From "2018 [YR2018]"', 'Filter by Color', and 'Number Filters'. The 'Clear Filter From "2018 [YR2018]"' option is selected. The spreadsheet shows a list of countries in column A and their corresponding values in column B. The values are: 6227, 6233, 6234, 7848.356445, 8272, 9212.432617, 10280.52637, and a blank cell. The blank cell is highlighted, indicating it is the missing data point.

Apply this to all columns to get clean data for the analysis stage.

#### 4. Analyse the data



#### 5. Communicate the data

This dataset is suitable for the government, especially the health ministry, to improve the quality of health services. With this dataset, we can do the most important initial stage, including finding information on countries' health expenditures worldwide.

We assume that a country's health expenditures are usually correlated with an increase in the quality of health services and broader community access.

We should analyse health care expenditure before further analysing the quality of health services and facilities.

In Figure 1, the top three countries with the most significant healthcare expenditures are the United States, Switzerland, and North America, while Australia is in 9<sup>th</sup> position. It is also crucial to analyse each country's population.

In Figure 2, the amount of health expenditure is not fully proportional to a country's population; for example, Switzerland has a population smaller than North America and the United States but a more significant healthcare expenditure than those countries.

I recommend further analysing the quality of health services and facilities in Switzerland, North America, and the United States.

## Challenge #2

### Step 1 - Download the Dataset

I have downloaded the data set 'the [customer personality analysis](#) dataset.

This dataset is marketing campaign data consisting of 29 features (columns). These 29 features can be grouped into four types of information: People, Products, Promotion, and Place.

People: is a feature or column that contains data about customers (ID, Year\_Birth, Education, Income, Kidhome, Teenhome, Dt\_Customer, Recency, and Complain).

Products: which is a feature or column that contains data about the type of product purchased by the customer (MntWines, MntFruits, MntMeatProducts, MntFishProducts, MntSweetProducts, MntGoldProds)

Promotion: which is a feature or column that contains data about the effectiveness of the campaign/promotion (NumDealsPurchases, AcceptedCmp1, AcceptedCmp2, AcceptedCmp3, AcceptedCmp4, AcceptedCmp5 and Response)

Type: is a feature or column that contains data about the type of customer purchase (NumWebPurchases, NumCatalogPurchases, NumStorePurchases and NumWebVisitsMonth).

### Step 2 - Prepare Micro scope of work

#### - Business Objective:

The scenario we get is to increase the revenue of a client named VINO Vineyard in wine sales.

From this dataset, one of the objectives that we can formulate is to determine better market segmentation by seeing if there is a special behaviour of customers of a certain age, a certain level of education or marital status that affects the purchasing pattern of certain products so that we can determine the target and more appropriate campaign methods to increase the number of product sales. Then, campaign data will also determine our decision to conduct campaigns for customers to reduce campaign costs.

#### - The data analysis SMART goal:

- Specific: We want to increase the number of products by finding a suitable market segmentation. We will look at this segmentation by analysing some of the features that may influence customer buying patterns, such as age, education level and marital status.
- Measurable: We will know this relationship by looking at blockchart visualisation and also the coefficient correlation visualisation of the features that we consider to influence product purchase.
- Achievable: This approach is considered the most realistic approach based on statistical theory known as 'Feature Engineering'.
- Relevant: This goal is very important because we will be able to recommend the target market and what type of campaign we should do because the type of campaign is highly correlated with the target market that we will target in the market.
- Time-bound: We will do this analysis using python or excel and finish within 3-7 days.

#### - A description of the work that will be completed:

- Some features that are considered to achieve the objectives that have been listed as: set are: Year\_Birth, Education, Marital Status, Dt\_Customer, MntWines, NumWebPurchases, NumCatalogPurchases, NumStorePurchases and NumWebVisitsMonth.
- Ignore features that we do not use in the analysis process to make it easier for us.
- Change the format of some features, such as Year\_Birth and Dt\_Customer, to get the customer's age and to find out how long ago the customer made the last purchase transaction.
- Calculate the distribution of the education level and marital status of the customer.

- If there are some feature values we can group to simplify the data, then do so. Marital\_Status which in the dataset has 8 classes (Married, Together, Single, Divorced, Widow, Alone, Absurd, YOLO) can be simplified into 2 (Married and Single).
  - Create a new feature by combining multiple features/columns whose information overlaps or can be combined. For example ('NumWebPurchases', 'NumCatalogPurchases', 'NumStorePurchases', 'NumDealsPurchases') can be combined into one feature/column.
  - Cleaning data that has duplicates or missing values.
  - Visualise the feature we want to see: the correlation coefficient, which shows the relationship between two or more variables.
- **A summary of how the existing data might help develop the future strategy.**  
This data set can help the Company determine marketing campaign strategies based on previous historical data.  
For example, the strategy to determine the target customer in marketing (market segment) for wine marketing can be based on the level of education or income and the number of members in the family.
  - **Any early-identified potential biases/risks/issues present in this dataset that could affect the analysis.**  
This includes the amount of missing data in the data set, which will result in less accurate analysis results. Some duplicated data will also impact the analysis results.
  - **Additional data that may need to be sourced (if applicable)**  
We need to get data about the varieties of wines in the market (from green grapes or red grapes), the brewing styles, the percentage of alcohol in wines (0%, 10% or any),
  - **Next step**  
For further analysis, we can cluster this dataset. Of course, with a clustering method such as K-means or other methods.

### Step 3 - Share your work & give feedback (optional)

I have shared my work and gave feedback.

## Challenge #3

### Step 1 - Access the dataset

I chose SQL to analyse the data so I accessed on Google BigQuery.

### Step 2 - Understand the data collection goal

The goal is to retrieve a dataset that can be used to identify countries with an old population of 65 years and above, which is the company's target market. It has been mentioned in the scenario that two things that should be found from the dataset are:

1. Identify the five most promising major markets
2. Identify five developing countries to which the company could potentially supply these devices.

From this goal, we must find some data, namely the population and income of each country.

### Step 3 - Collect the relevant data

The relevant data to answer the objectives are:

3. bigquery-public-data.world\_bank\_global\_population.population\_by\_country
4. bigquery-public-data.world\_bank\_wdi.country\_summary.

### Step 4 - Collate your report

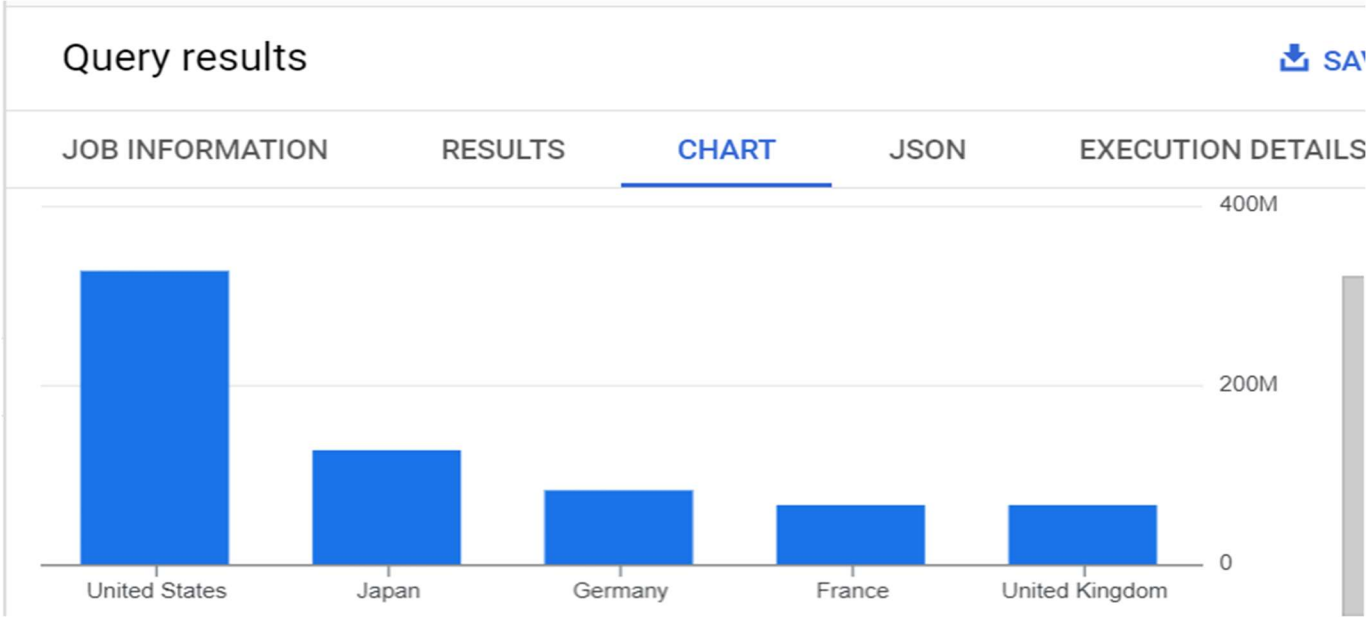
I took 5 features/variables: 3 from the country summary dataset and 2 from the population dataset. The query retrieves the top 100 countries, along with their population for 2018, special notes, and income groups. This query ensures that only countries with valid (non-zero) income groups are included, and the results are sorted by most population.

```
SELECT
  pop.country,
  country.country_code,
  country.special_notes,
  country.income_group,
  pop.year_2018
FROM `bigquery-public-data.world_bank_wdi.country_summary` AS country
LEFT JOIN `bigquery-public-data.world_bank_global_population.population_by_country`
AS pop
ON country.country_code = pop.country_code
WHERE country.income_group IS NOT NULL
ORDER BY pop.year_2018 DESC
LIMIT 100;
```

If we want to do a more specific search for 5 developing countries, we can change the query by specifying the search income = High income because the income of developing countries is identical to that of their population, which is in the high category.

```
SELECT
  pop.country,
  country.country_code,
  country.special_notes,
  country.income_group,
  pop.year_2018
FROM `bigquery-public-data.world_bank_wdi.country_summary` AS country
LEFT JOIN `bigquery-public-data.world_bank_global_population.population_by_country` AS pop
ON country.country_code = pop.country_code
WHERE country.income_group IN ('High income')
ORDER BY pop.year_2018 DESC
LIMIT 5;
```

The five developing countries with the most promising large markets are the United States, Japan, Germany, France, and the United Kingdom.



## Challenge #4

### 1. Access Dataset

I use Python, the programming language from the Google collaboratory platform, to manipulate data. Because the dataset is on kaggle.com, which is open source, we can directly access it from Google Colab with the following steps:

- Install 2 library :

```
!pip install opendatasets
!pip install pandas
```

- Import library

```
import opendatasets as od
import pandas as pd
```

- Read the dataset using the link source dataset from Kaggle

```
od.download("https://www.kaggle.com/datasets/datafiniti/womens-shoes-prices")
```

After the code is run, enter the Kaggle username and Kaggle key that can be obtained from the Kaggle account.

- Read datasets in the form of dataframes gar easier to manipulate

```
df = pd.read_csv("/content/womens-shoes-prices/Datafiniti_Womens_Shoes_Jun19.csv")
```

### 2. Data manipulation to get the answer

- What is the range of shoe prices?

Firstly, we filter our dataset only to retrieve information related to shoe products.

```
shoes = df[df["primaryCategories"]=="Shoes"]
```

Then, we can search for the price range of shoes by looking at the minimum and maximum price using min-max in Python.

```
print("minimum price : ", min(shoes["prices_amountMax"]))
print("maximum price : ", max(shoes["prices_amountMax"]))
```

The shoe price range was found to be 5.5 USD - 5000 USD.

- What are the median and average prices for women's shoes ?

```
median_price = shoes["prices_amountMax"].median()
average_price = shoes["prices_amountMax"].mean()
print("Median price:", median_price)
print("Average price:", average_price)
Obtained Median Price = 51.99 USD and Average Price = 72.45 USD
```

- How many unique shoe models does it contain?

```
category_counts = shoes['manufacturer'].value_counts()
print(category_counts)
category_counts = shoes['brand'].value_counts()
print(category_counts)
category_counts = shoes['colors'].value_counts()
print(category_counts)
```

In the shoe category in this dataset, if the uniqueness of shoes is seen from the manufacturer, there are 57 types; if seen from the brand, there are 505 different brands. Then, when viewed from the uniqueness of the colour, there are 1907 kinds of colours.

- How many different systems are shoe sizes encoded in this database?

```
category_counts = shoes['sizes'].value_counts()
print(category_counts)
491
```

- For how long has the database been maintained?  
(as in, how many months are there between the first entry in the database and the latest entry?)  
Because the date column dataset is still an object type, we need to change the dataset format to datetime format. After the data format is appropriate, we can find how long this database has been maintained by looking for the initial date the shoe data was entered and the last data entered in the database.

```
shoes['dateAdded'] = pd.to_datetime(shoes['dateAdded'])
shoes.sort_values('dateAdded')
min_date = shoes['dateAdded'].min()
max_date = shoes['dateAdded'].max()
duration_in_months = (max_date.year - min_date.year) * 12 + (max_date.month -
min_date.month)
print(duration_in_months)
```

The initial date of data entry was 2015-01-01, and the last date was 2019-01-01, so the database has been maintained for 48 months.

- How can this dataset be useful for the business problem at hand?  
Of course, there are many things we can do with this dataset, including analysing what brands are selling the most, and then we can find what kind of shoe colour patterns are in demand the most. This insight can be an idea for shoe manufacturers with the most sales to produce shoes that suit market tastes and continue to dominate the market. This can also be an advantage for competitors to make shoes that can compete with shoe manufacturers who have been dominating the market.
- A unique insight or inspiration that you have taken from the dataset.  
One insight I can gain from the data is that each shoe brand may have different market segments. This can be seen when I try to find the five brands with the most sales and the five brands with the highest prices.

```
category_counts = shoes['brand'].value_counts()
category_price = shoes[shoes['prices_amountMax']>4000]['brand'].value_counts()
top_5_brands_selling = category_counts.head(5)
print(top_5_brands_selling)
top_5_brands_expensive = category_price.head(5)
print(top_5_brands_expensive)
```

The five brands with the most sales are Brinley Co., SAS, Trotters, Propet, and Soda, while the five brands with the highest selling prices are Puma, Trotters, Softwalk, Stev-madden and Faux Suede. Four out of five have different segments. Some brands target high sales numbers and brands that target high selling prices. Usually, brands that target high sales use the concept of scarcity because the number of products produced is minimal, a 'limited edition', so the price is very high (information on the dataset is sold at 5000 USD).



## CHALLENGE #5

### 1. Download Dataset

We use the Python programming language with the Google Colab platform, so the dataset from Kaggle can be directly accessed because it is open source.

This dataset consists of two files, namely beer data and breweries data. Each dataset has different features along with the features and feature descriptions:

**Beer Dataset:** Name (Name of the beer), Style (Beer style), Abv (The alcoholic content by volume with 0 being no alcohol and 1 being pure alcohol), Brewery\_id (Unique identifier for brewery that produced this beer), Ounces (Size of beer in ounces), Ibu (International bittering units, which describe how bitter a drink is)

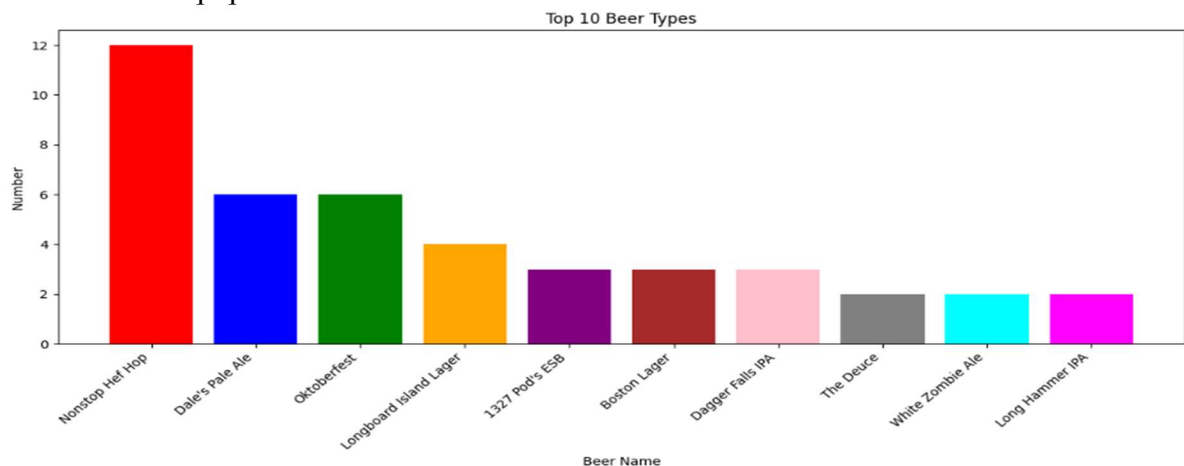
**Breweries Dataset:** Brewery\_id (Unique identifier for the brewery that produced this beer), Name (Name of the brewery), City (City that the brewery is in), State (State that the brewery is located).

### 2. Some Questions from the Dataset

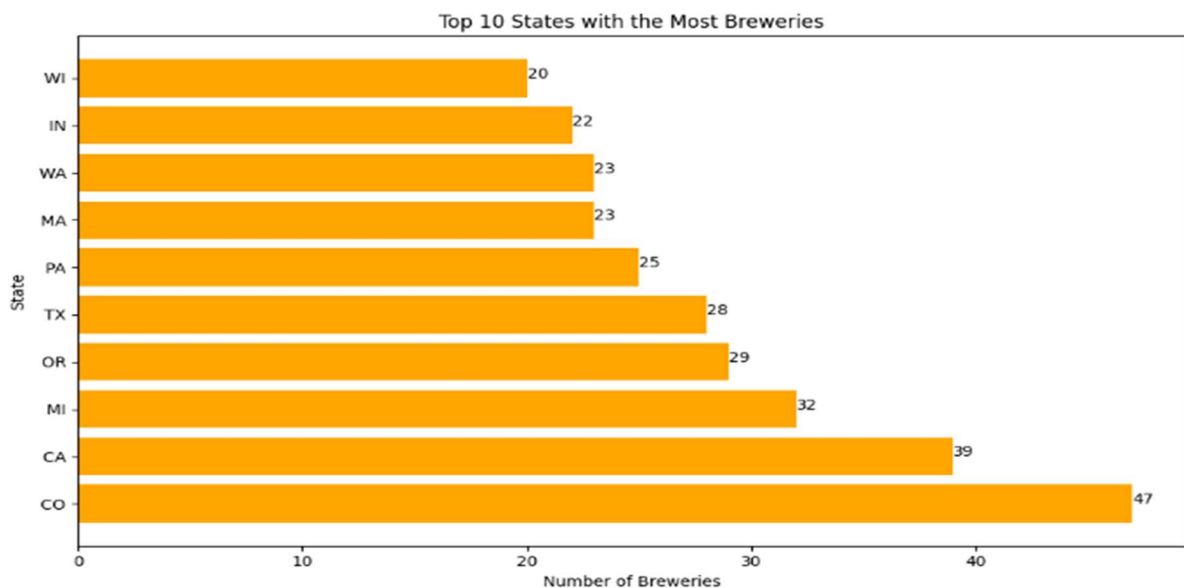
- What is the most popular beer?
- What countries have the most breweries?
- What beer styles are popular and what is the average alcohol content?
- Are some beers very bitter & some have high alcohol content?
- How is the Distribution of Alcohol Content in Beer

### 3. Exploratory charts

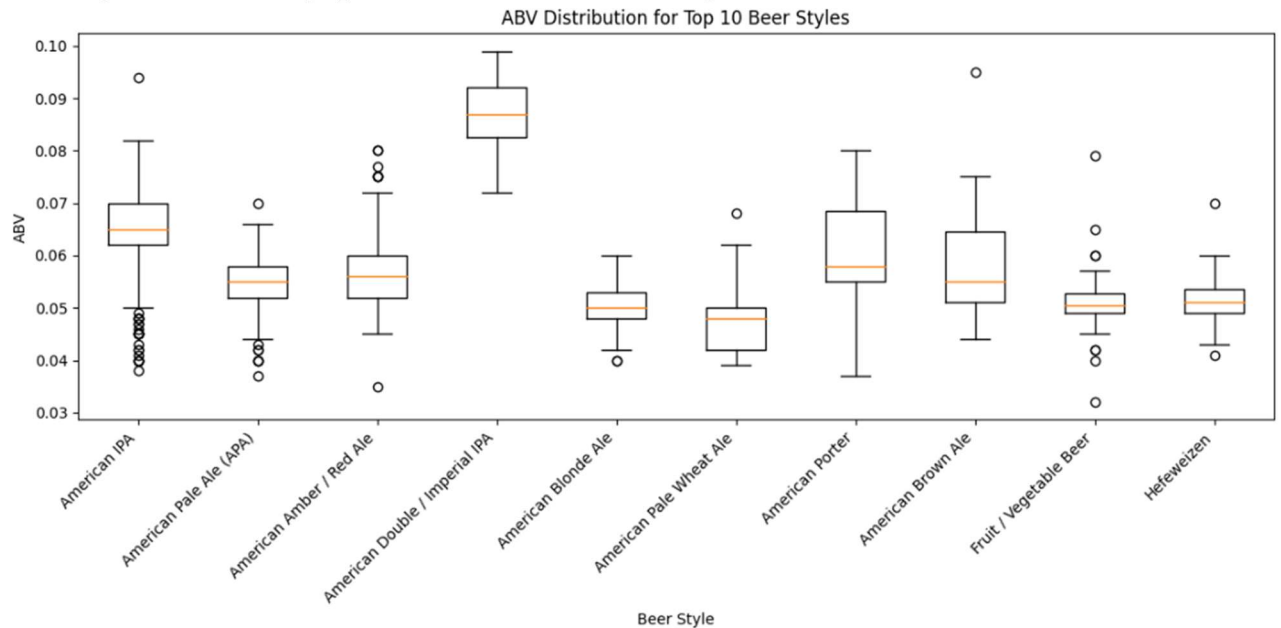
- What is the most popular beer?



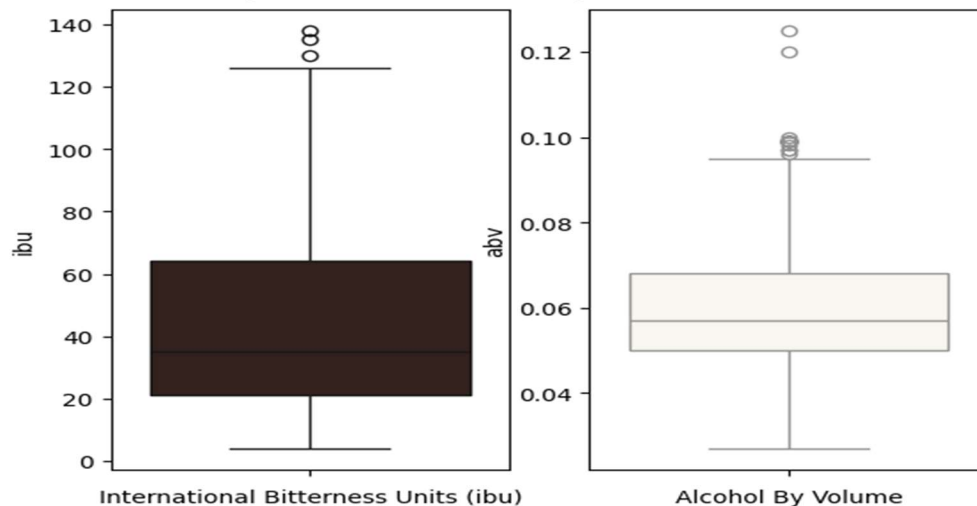
- What countries have the most breweries?



- What styles of beer are popular and what is their average alcohol content?



- Are some Beers very bitter & some have a high alcohol content?



#### 4. Findings dan Insights

- The most popular beer type is Nonstop Hef Hop.
- The state with the most breweries is Colorado, which has 47 breweries.
- The most popular beer style is American IPA, with an average alcohol content of 0.065 (6.5%).
- Most beers have a bitterness level (IBU) of around 40 and an alcohol level (ABV) of around 6%. The range of IBU values was more varied than the ABV, indicating more significant variation in bitterness than alcohol content in the analysed beers.

#### 5. Insights

In the first analysis, we found that nonstop hef hop is the most popular beer, so this product will be the strongest competitor when we make beer products.

The second analysis then reveals some places with the most breweries, which we can visit to find beer suppliers or learn about beer types.

Furthermore, the third and fourth analyses provide more specific information, namely the most popular beer styles and their alcohol content, which can be our basis when making beer that can compete in the beer market.

## CHALLENGE #6

I process data using Python, which is read in data frame format to facilitate processing. Some libraries used are Pandas, Numpy, Seaborn, and Matplotlib. After the data is read, we first check this dataset's feature/column name because naming the feature/column is one of the most important things in Python before cleaning.

Many feature names still do not match the Python format, which recommends using feature names without spaces or punctuation marks other than (\_). So, the first thing we do is change the feature name and delete some features that we don't need.

### Changing the feature name format

We change the feature/column naming format using Python's rename method.

**Before:**

**After:**

|   | Q1:<br>GOING<br>OUT? | Q2:<br>GENDER | Q3:<br>AGE | Q4:<br>COUNTRY | Q5:<br>STATE,<br>PROVINCE,<br>COUNTRY,<br>ETC |   | going_out | sex  | age | country | state    |
|---|----------------------|---------------|------------|----------------|-----------------------------------------------|---|-----------|------|-----|---------|----------|
| 0 | NaN                  | NaN           | NaN        | NaN            | NaN                                           | 0 | NaN       | NaN  | NaN | NaN     | NaN      |
| 1 | No                   | Male          | 44         | USA            | NM                                            | 1 | No        | Male | 44  | USA     | NM       |
| 2 | NaN                  | Male          | 49         | USA            | Virginia                                      | 2 | NaN       | Male | 49  | USA     | Virginia |

Then, we move to the next step, which is data cleaning.

### Data Cleaning

We first check the NaN value using the code `df.isnull().sum()`.

It can be seen that this dataset is very messy and needs a lot of cleaning; the data has many missing values (NaN). Then we delete columns that have too many NaN values. The column that is deleted here is the column that has more than 80% NaN values from the total data.

|     | Column                    | NaN Count | Percentage NaN |
|-----|---------------------------|-----------|----------------|
| 113 | Unnamed: 113              | 2451      | 99.634146      |
| 118 | yahoo                     | 2393      | 97.276423      |
| 115 | dailydish                 | 2375      | 96.544715      |
| 117 | espn                      | 2361      | 95.975610      |
| 111 | Q9: OTHER COMMENTS        | 2074      | 84.308943      |
| 110 | despair_other             | 1743      | 70.853659      |
| 109 | joy_other                 | 1549      | 62.967480      |
| 116 | science                   | 1098      | 44.634146      |
| 45  | Q6   JoyJoy (Mit Iodine!) | 1026      | 41.707317      |
| 58  | Q6   Maynards             | 1024      | 41.626016      |

We can see that the column/feature to be deleted is Unnamed:113, Yahoo (Q12), Daily Dish (Q12), ESPN (Q12), and Q9:Other Comments.

- **Q1: Going\_Out.** At the column going\_out there are 3 values including “NaN”, “No”, and “Yes”. The value of NaN can be replace with “Maybe”.
- **Q2:Gender.** Before. It can be seen that there are 5 unique values in gander, here it is chosen to replace the values ‘I'd rather not say’ and ‘Other’ with ‘Didn't say’. Then the NaN value is also filled with ‘Didn't say’.After
- **Q3: Age**  
Before. It can be seen that there are non-numeric values where the age column should contain numeric values. So the first thing to do is to replace the non-numeric values with NaN values. After that, all NaN values will be replaced with the Median value from the age column.

```
df['age'].unique()

array([nan, '44', '49', '40', '23', '53', '33', '43', '56', '64', '37',
       '59', '48', '54', '36', '45', '25', '34', '35', '38', '58', '50',
       '47', '16', '52', '63', '65', '41', '27', '31', '61', '46', '42',
       '62', '29', '39', '32', '28', '69', '67', '30', '22', '26', '51',
       '70', '24', '18', '19', 'Old enough', '57', '60', '66', '12',
       'Many', '55', '72', '?', '21', '11', 'no', '9', '68', '20', '6',
       '10', '71', '90', '13', '45-55', '312', '99', '7', 'hahahahaha',
       '88', '39.4', '74', '102', 'older than dirt', '17', '15', '8',
       '75', '5u', 'Enough', 'See question 2', '24-50', '14', 'Over 50',
       '100', '76', '1000', 'sixty-nine', '46 Halloweens.', 'ancient',
       '77', 'OLD', 'old', '73', '70 1/2', '1', 'MY NAME JEFF', '4',
       '59 on the day after Halloween', 'old enough', 'your mom',
       'I can remember when Java was a cool new language', '60+'],
      dtype=object)
```

After

```
df['age'].unique()
```

```
array([41, 44, 49, 40, 23, 53, 33, 43, 56, 64, 37, 59, 48, 54, 36, 45, 25,
       34, 35, 38, 58, 50, 47, 16, 52, 63, 65, 27, 31, 61, 46, 42, 62, 29,
       39, 32, 28, 69, 67, 30, 22, 26, 51, 70, 24, 18, 19, 57, 60, 66, 12,
       55, 72, 21, 11, 68, 20, 10, 71, 13, 74, 17, 15, 14, 73])
```

#### • Q4:Country

In the country column are 64 NaN values and several wrong country names or the name of a country with different names such as USA, America, U.S.A. To clean up the wrong naming, it is necessary to standardise the writing of country names in the dataset. First, we changed the entire country column value to lowercase letters. Using fuzzy matching technique from fuzzywuzzy library to correct variations in writing or typos. Of course, we need to list country names that we will refer to checking. Meanwhile, the NaN value will replace the word 'other'. Then, because we are asked to filter between America and not, we can replace countries other than America with 'not America'.

```
df['country'].unique()
```

```
array(['other', 'usa', 'canada', 'uk', 'mexico', 'france', 'netherlands',
       'germany', 'australia', 'ireland', 'japan', 'scotland',
       'switzerland', 'china'], dtype=object)
```

| country     | count |
|-------------|-------|
| usa         | 2062  |
| not america | 398   |

dtype: int64

#### • Q5: State.

In the state column, the value is also checked with print(df['state'].nunique()), and the result is 511. Because the unique value in the state is too diverse and the state column is represented by country, it can be removed. The same process can be done on the country column.

#### • Q6 all columns.

Q6 is a column with types of candies and contains data on people's reviews of these candies. Each column contains four unique values: 'Meh', 'Despair', 'Joy', and NaN. The NaN value in all Q6 columns can be replaced with 'Unknown', which is considered an unknown reaction.

#### • Q7:Joy\_Other and Q8:Despair\_Other. The data contents of these two columns are long sentences.

```
df['joy_other'].unique()
```

```
array([nan, 'Mounds',
       "Reese's crispy crunchy bars, 5th avenue bars, ferrero rocher, dries fruit other than raisins",
       'using my elbows to get in front of the little kids in line.',
       'Peanut butter m&ms ', 'Quality brand dark chocolate',
       'Cheesy Snax', 'Cadbury Fingers', 'Sour keys',
       'Payday, baby Ruth ', 'Any full size candy bar', 'Beer',
       'Mounds, Almond Joy (it's in the name, for god's sake)',
       'Chunky, Marathon Bar, Caramello, Almond Joy, Mounds, Charleston Chew',
       'Puppies, pure sunshine, kombucha, and a kiss from my one true love',
       'Jelly Belly jelly beans!',
       'Cow Tails! Butterscotch! Sugar Babies!', 'Puppies',
       'Marathon Bars (in England they are called Curly Wurlies). Pop rocks & Space dust, Almond Joy',
       'Almost anything from Lindt', 'SIXLETS, WHOPPERS, ZERO',
       'raw eggs and a map of neighborhood houses not giving out candy',
       'A cappuccino',
       'Any chocolate covered fruit, those orange-flavored chocolate orbs that you crack into slices, chocolate covered pretzels',
       'hot tamales or other cinnamon candies',
       'Exotic international candies',
       'Belly Flops (Irregular Jelly Belly from the Dollar Store)',
       'Brussels sprouts, canned spinach, canned SPAM', 'Tootsie Rolls',
       'Warheads', 'Almond Joy, Peanut Butter M&Ms, Bullseyes',
       'Family, granddaughter',
```

we use the Counter library from collections and then take the three most important words.

```
from collections import Counter
def find_important_words(text):
    if pd.isnull(text):
        return ""
    words = text.lower().split()
    word_counts = Counter(words)
    most_common_words = [word for word,
count in word_counts.most_common(3)]
    return ','.join(most_common_words)

df['important_joy_words'] =
df['joy_other'].apply(find_important_words)
```

```
df['important_joy_words']
```

|      | important_joy_words  |
|------|----------------------|
| 0    |                      |
| 1    | mounds               |
| 2    |                      |
| 3    | bars,,reese's,crispy |
| 4    |                      |
| ...  | ...                  |
| 2455 | mounds               |
| 2456 |                      |
| 2457 | tiny,bottles,of      |
| 2458 |                      |
| 2459 |                      |

## CHALLENGE #7

### 1. Download Dataset

I used python programming language with Google Colab platform, so the dataset from kaggle can be directly accessed because it is open source. This dataset consists of two files, namely beer data and breweries data.

### 2. Join two data

These two datasets have one column/feature that is related from the scraping results, namely 'brewery\_id'. Here I use the left join technique from pandas python.

```
df_merged = pd.merge(df_beer, df_breweries, on='brewery_id', how='left')
df_merged.head()
```

```
df_merged = pd.merge(df_beer, df_breweries, on='brewery_id', how='left')
df_merged.head()
```

| Unnamed: 0 | abv | ibu   | id  | name_x |                     | style                          | brewery_id | ounces | name_y                    | city | state |
|------------|-----|-------|-----|--------|---------------------|--------------------------------|------------|--------|---------------------------|------|-------|
| 0          | 0   | 0.050 | NaN | 1436   | Pub Beer            | American Pale Lager            | 408        | 12.0   | 10 Barrel Brewing Company | Bend | OR    |
| 1          | 1   | 0.066 | NaN | 2265   | Devil's Cup         | American Pale Ale (APA)        | 177        | 12.0   | 18th Street Brewery       | Gary | IN    |
| 2          | 2   | 0.071 | NaN | 2264   | Rise of the Phoenix | American IPA                   | 177        | 12.0   | 18th Street Brewery       | Gary | IN    |
| 3          | 3   | 0.090 | NaN | 2263   | Sinister            | American Double / Imperial IPA | 177        | 12.0   | 18th Street Brewery       | Gary | IN    |
| 4          | 4   | 0.075 | NaN | 2262   | Sex and Candy       | American IPA                   | 177        | 12.0   | 18th Street Brewery       | Gary | IN    |

We can see that the two datasets have become one dataset. Some column names changed from the previous data, namely 'name' in the beer dataset to 'name\_x' (beer brand name) and 'name' in the breweries dataset to 'name\_y' (breweries name).

### 3. Discover cities and Breweries to visit

- *The two most common styles of beers on the list*

By counting the number of unique values of each beer style.

```
style_counts = df_merged['style'].value_counts()
two_most_common_styles = style_counts.head(2)
print(two most common styles)
```

It was found that the two most common beer styles were **American IPA** with 424 and **American Pale Ale (APA)** with 245.

- *The city which produces most of brands of beers (from two common styles of beers)*

By filtering the dataset using the keywords American IPA and American Pale Ale (IPA) then calculating the unique value of grouping city names and beer brands using the groupby method in python. Here is the code;

```
american_ipa_df = df_merged[df_merged['style'] == 'American IPA']
american_apale_df = df_merged[df_merged['style'] == 'American Pale Ale (APA)']
city_brand_counts_ipa = american_ipa_df.groupby('city')['name_x'].nunique()
city_brand_counts_apale = american_apale_df.groupby('city')['name_x'].nunique()
print(city_brand_counts_ipa.idxmax())
print(city_brand_counts_apale.idxmax())
```

It was found that the cities that produce the most beer brands of the two styles are **Tampa** and **Chicago**.

- *The most common styles of beers produced for particular state (number of brands produced, average ABV and IBU)*

Firstly, I'm looking at the two states that produce the most types of beer brands: California (CA) and Colorado (CO). I then analysed to find the most common beer styles produced from these two countries and looked for information related to the number of brands produced and also the average ABV and IBU of these beer styles.



| Colorado (CO)                    | California (CA)                  |
|----------------------------------|----------------------------------|
| style                            | style                            |
| American IPA 40                  | American IPA 45                  |
| American Pale Ale (APA) 40       | American Pale Ale (APA) 17       |
| Name: count, dtype: int64        | Name: count, dtype: int64        |
| Style: American IPA              | Style: American IPA              |
| Number of Unique Beer Names: 36  | Number of Unique Beer Names: 42  |
| Average IBU: 70.34782608695652   | Average IBU: 66.6842105263158    |
| Average ABV: 0.06613157894736842 | Average ABV: 0.06711111111111111 |
| -----                            | -----                            |
| Style: American Pale Ale (APA)   | Style: American Pale Ale (APA)   |
| Number of Unique Beer Names: 37  | Number of Unique Beer Names: 16  |
| Average IBU: 56.4                | Average IBU: 40.92307692307692   |
| Average ABV: 0.05876923076923076 | Average ABV: 0.05152941176470586 |
| -----                            | -----                            |

- **The city or state which produced the most distinct styles of beers.**

The most distinct type of beer style means a rare or one-of-a-kind style. Here, we first look for unique beers. Then, we make all types of beer styles into a list dictionary based on the country. After that, using a for loop, we search for beer styles from a country that are not owned by other countries.

| abv  | ibu   | id   | name_x | style                | brewery_id         | ounces | name_y | city                     | state           |
|------|-------|------|--------|----------------------|--------------------|--------|--------|--------------------------|-----------------|
| 2050 | 0.063 | 23.0 | 1366   | Stupid Sexy Flanders | Flanders Oud Bruin | 25     | 16.0   | Sun King Brewing Company | Indianapolis IN |

The search results show that the rarest beer style is Flanders Oud Bruin from the state of Indiana, city of Indianapolis.

- **Top 5 cities and breweries wish list.**

From all of this information, the 5 best cities and 5 best breweries that I will visit are:

#### City:

- Tampa (City with the most American IPA beer styles)
- Chicago (City with the most American Pale Ale (APA) beer styles)
- Indianapolis (City with the most distinct beer styles with only one, Flanders Oud Bruin.
- Portland (the city with the most breweries)
- Grand Rapids (the city with the most beer brands)

#### Breweries:

- Brewery Vivant (Breweries with the most brand beer production of all breweries)
- Oskar Blues Brewery (Breweries with the most brand beer production in Colorado)
- 21st Amendment Brewery (Breweries with the most brand beer production in California)
- Portside Brewery (Breweries with the highest ABV)
- Sun King Brewing Company (Brewer of the rarest Flanders Oud Bruin beer style)

## CHALLENGE #8

### 1. Download Dataset

The dataset is taken directly from Kaggle (using the Kaggle API account) via Google Collab, using the Python programming language.

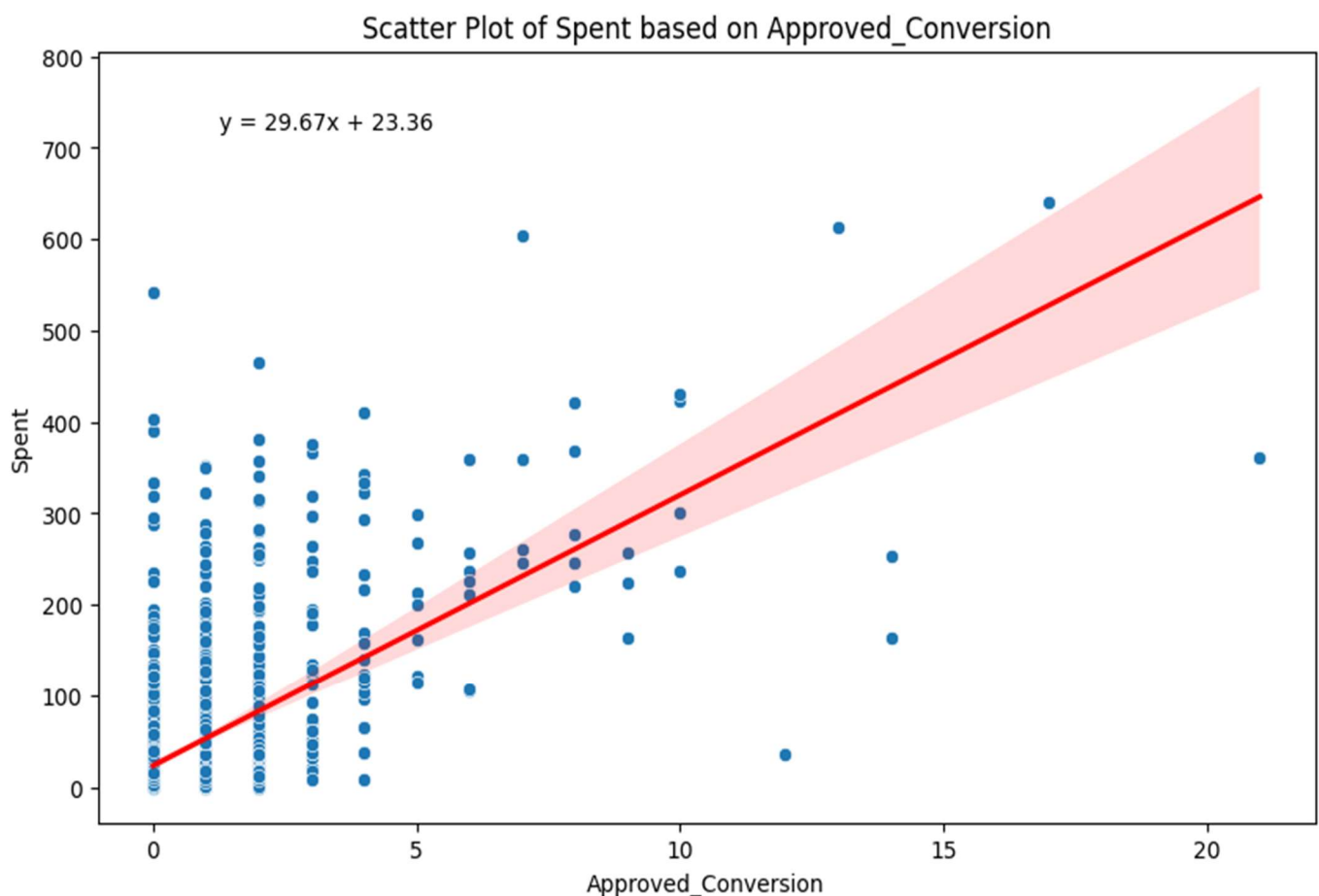
### 2. Create a campaign performance report

- The overall average cost per approved conversion is **33.90%**  
Build A simple linear regression model to predict approved conversion based on money spent.

How much will each approved conversion cost?

To create a linear regression model in python we can use the library from sklearn. The independent variable is Approved\_Conversion and the dependent variable is Spent.

The two parameter values that will determine this model are regression coefficient and regression intercept. After the data is trained into a linear regression model, we get the regression coefficient of **29.66674802** and the regression intercept of **23.35503836**.



By using regression coefficient and regression intercept, we can predict spent based on approved conversion with the following formula

$$\text{Approved Conversion} = (\text{Regression coefficient} \times \text{Spent}) + \text{Regression Intercept}$$

$$\text{Approved Conversion} = (29.66674802 \times \text{Spent}) + 23.35503836$$

So it is found that every approved conversion cost of **±53.02 AUD**.

- Why might this differ from the average cost calculated above?  
The values of the two are different because the average uses a simple calculation that only divides the sum of all variable values by the number of observations without paying attention to the relationship between the two. In contrast, linear regression pays close attention to the relationship between the two. The conclusion is that averages provide an overview of the centre of the data distribution, while linear regression describes the relationship between two variables.
- Which of the values might be more representative of predictions?  
The more representative answer is the value of the linear regression model. A linear regression model is used when we want to understand the relationship between two variables and how one variable affects the other. It is very useful for prediction and causal analysis.